ASPLOS 2026

Evaluating Compiler Optimization Impacts on zkVM Performance

Thomas Gassmann
ETH Zurich
Switzerland

Stefanos Chaliasos
UCL Centre for Blockchain Technologies
United Kingdom
zkSecurity
USA

Thodoris Sotiropoulos
ETH Zurich
Switzerland

Zhendong Su
ETH Zurich
Switzerland

Wonjae Choi

2026. 04. 24

Virtual Machine & Optimization Laboratory

Dept. of Electrical and Computer Engineering

Seoul National University

Outline

I. Background

II. Motivation

III. Methodology

IV. Key Findings

V. zkVM-Aware Optimizations

VI. Conclusion & Discussion

Background

Zero-Knowledge Proofs & zkVMs

Zero-Knowledge Proof (ZKP)

- Prover convinces a verifier that a statement is true, revealing nothing else
- Enables privacy-preserving, verifiable computation (blockchains, credentials, ...)

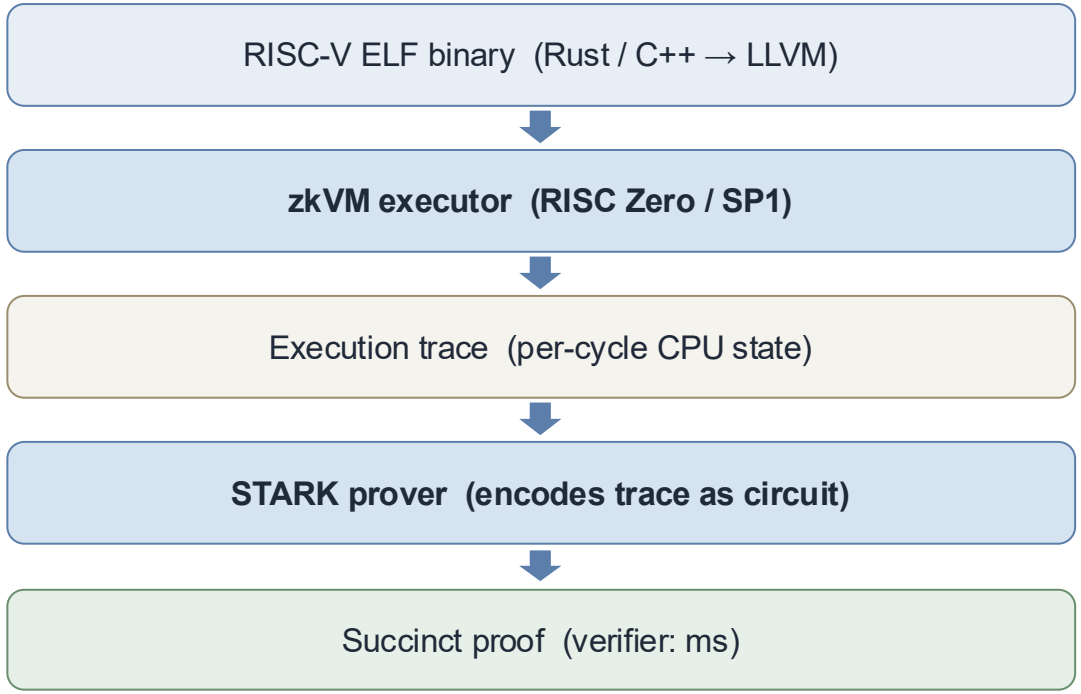
Zero-Knowledge Virtual Machine (zkVM)

- Write programs in Rust/C++ → compile to RISC-V → ZKP generated automatically
  - Removes the need to hand-craft arithmetic circuits
- Production zkVMs studied: RISC Zero, SP1 (both RISC-V based)

Key tension

- zkVMs inherit LLVM, a compiler tuned for traditional CPUs — is that optimal?

How a zkVM Works




```
cycle  pc         instr            regs / stack / mem
  0    0x20000    addi x1,x0,5     x1 ← 5
  1    0x20004    addi x2,x0,7     x2 ← 7
  2    0x20008    mul  x3,x1,x2    x3 ← 35
  3    0x2000c    sw   x3,0(x4)    M[x4] ← 35
  …    …          …                …
```

1. Execute

- Run the RISC-V binary in the zkVM executor
- Each cycle appends a row to the trace

2. Encode the trace

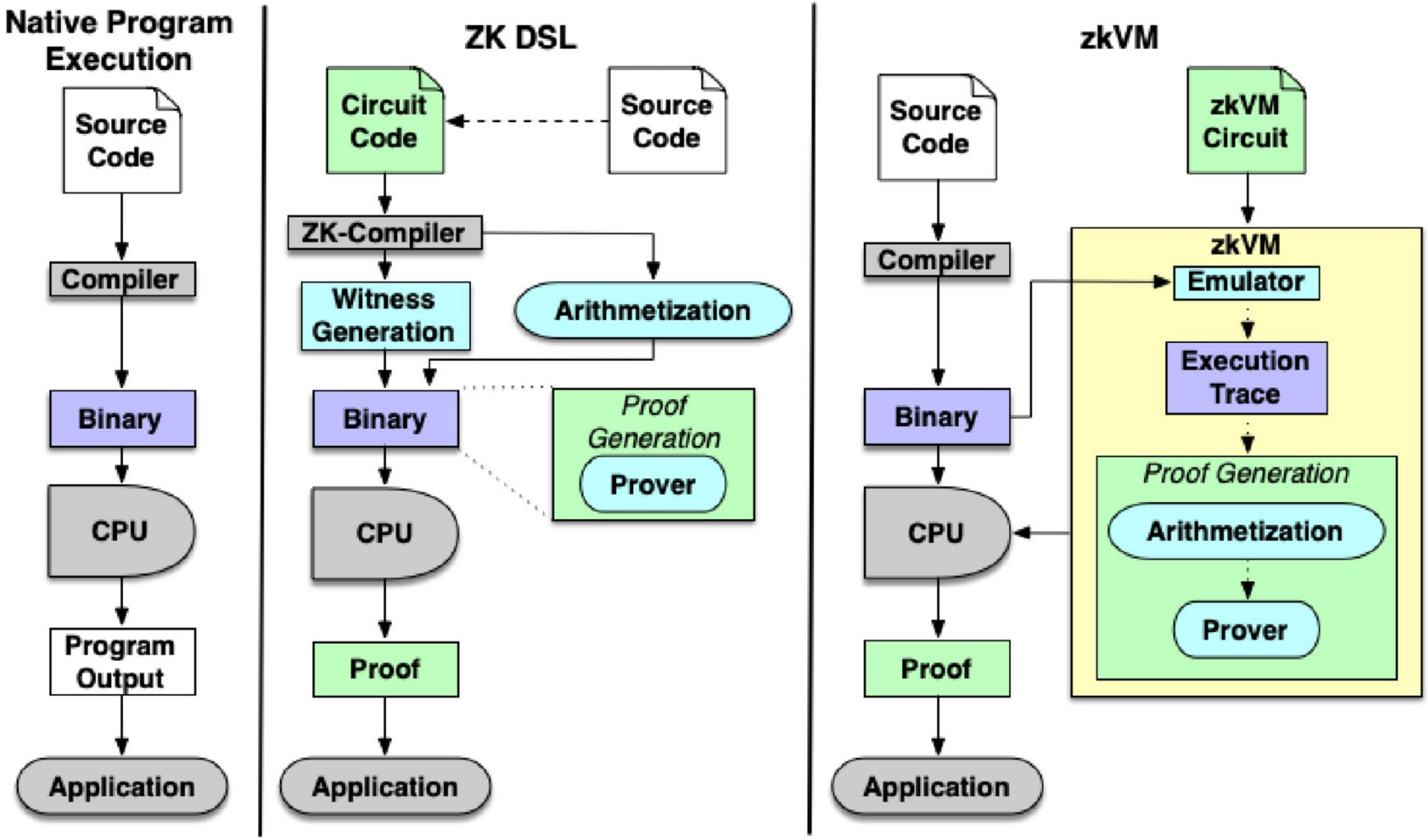
- Each row must satisfy the RISC-V transition function
- Memory consistency is enforced across rows

3. Prove & verify

- STARK attests the trace is a valid run of the committed binary
- Verifier checks in ms — no re-execution

Proving cost scales with trace length
fewer instructions ≈ faster proofs

Architecture comparison



Motivation

Optimizations with Negative Effects on zkVMs

```
// unoptimized version
example::div:
        li t0, 8
        div a0, a0, t0
        ret
```

```
// optimized version
example::div:
        srai a1, a0, 31
        srli a1, a1, 29
        add a0, a0, a1
        srai a0, a0, 3
        ret
```

(a) CPU-oriented instruction selection can hurt zkVMs: on RISC Zero, the 'optimized' form takes 8 cycles vs. 4 unoptimized

```
/* unoptimized version */
int i, a[N], b[N];
for (i = 0; i < N; i++) {
    a[i] = 1;
    b[i] = 2;
}
```

```
/* optimized version */
int i, a[N], b[N];
for (i = 0; i < N; i++) {
    a[i] = 1;
}
for (i = 0; i < N; i++) {
    b[i] = 2;
}
```

(b) *Loop fission* often helps CPUs but can hurt zkVMs by duplicating loop-control work. N is set to 1048576.

Why zkVMs Break Standard Optimizations

zkVMs differ fundamentally from traditional CPUs

- Uniform instruction cost — no fast vs slow instructions
- No cache, no branch prediction, no ILP, no out-of-order execution
- Memory paging is the dominant variable cost (~1130 cycles per page-in/out)
- Floating-point emulated, no multithreading

LLVM optimizations were tuned for the opposite assumptions

- Cost models assume hardware features zkVMs don't have
- Heuristics target cache locality, branch prediction, ILP — irrelevant for proofs
- Many transformations trade more instructions for cheaper ones — the wrong trade-off here

Research question

- How much do existing LLVM optimizations actually help (or hurt) zkVMs?

Two Motivating Examples

Strength reduction (integer division)

- LLVM replaces `div` with shift + add — 3× faster on x86
- On RISC Zero: 4 cycles (`div`) → 8 cycles (shift+ add) — 2× slower
- All zkVM instructions cost the same; trading one op for many is a net loss

```
// unoptimized version
example::div:
        li t0, 8
        div a0, a0, t0
        ret
```

```
// optimized version
example::div:
        srai a1, a0, 31
        srli a1, a1, 29
        add a0, a0, a1
        srai a0, a0, 3
        ret
```

Loop fission (splitting one loop into two)

- On x86: improves cache locality, enables vectorization → faster
- On zkVMs: duplicates loop counter and branch — extra trace rows, no benefit
- No cache to exploit; every extra instruction directly costs cycles

```
/* unoptimized version */
int i, a[N], b[N];
for (i = 0; i < N; i++) {
    a[i] = 1;
    b[i] = 2;
}
```

```
/* optimized version */
int i, a[N], b[N];
for (i = 0; i < N; i++) {
    a[i] = 1;
}
for (i = 0; i < N; i++) {
    b[i] = 2;
}
```

Takeaway

- CPU-tuned heuristics can quietly hurt zkVM performance

Methodology

Experimental Setup

Three research questions

- RQ1: How does each individual LLVM pass affect zkVM performance?
- RQ2: What can autotune combinations of passes achieve?
- RQ3: How do results compare to traditional CPUs (x86)?

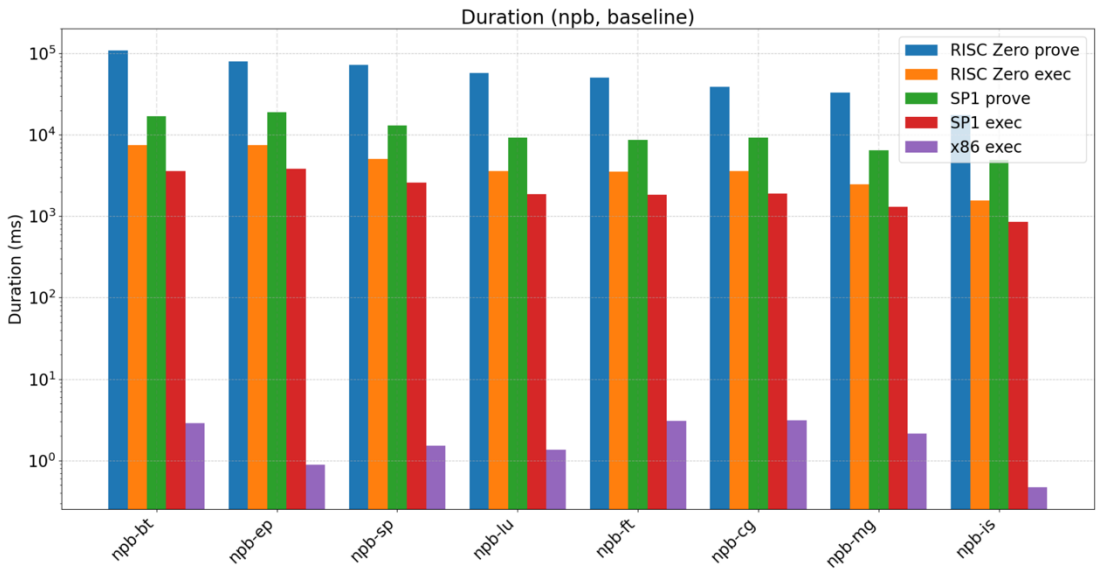
Experimental scope

- 71 optimization profiles: 64 individual LLVM passes + 6 levels (`-O0`..`-Oz`) + baseline
- 58 benchmarks: PolyBench, NPB, cryptography (SHA, Keccak, ECDSA, ...), SPEC
- Two zkVMs: RISC Zero 1.2.4 and SP1 4.1.4 (both RISC-V, STARK-based)

Metrics

- Cycle count · zkVM execution time · proving time

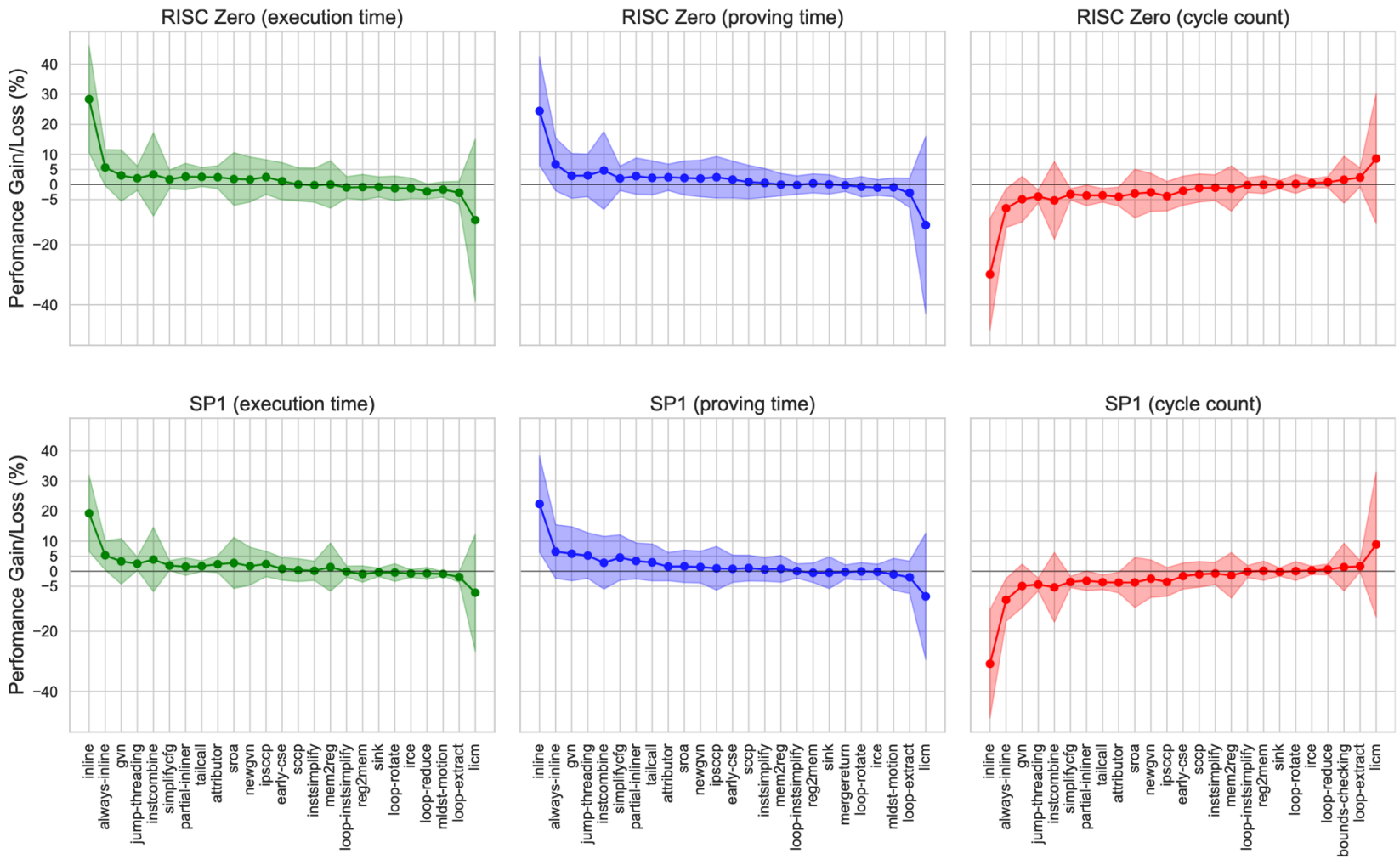
Execution & Proving Time



zkVM	Execution				Proving			
	Min	Max	Mean	Median	Min	Max	Mean	Median
RISC Zero	0.04	157.70	4.51	0.34	0.53	2071.24	60.85	3.83
SP1	0.06	41.81	1.70	0.23	0.38	205.87	8.89	1.90

Key Findings

RQ1 · Impact of Individual LLVM Passes



RQ1 · Impact of Individual LLVM Passes

Most beneficial passes

- `inline`, `always-inline`, `gvn`, `jump-threading`, `instcombine`, `simplifycfg`
- Gains typically < 10% per pass, but consistent across benchmarks

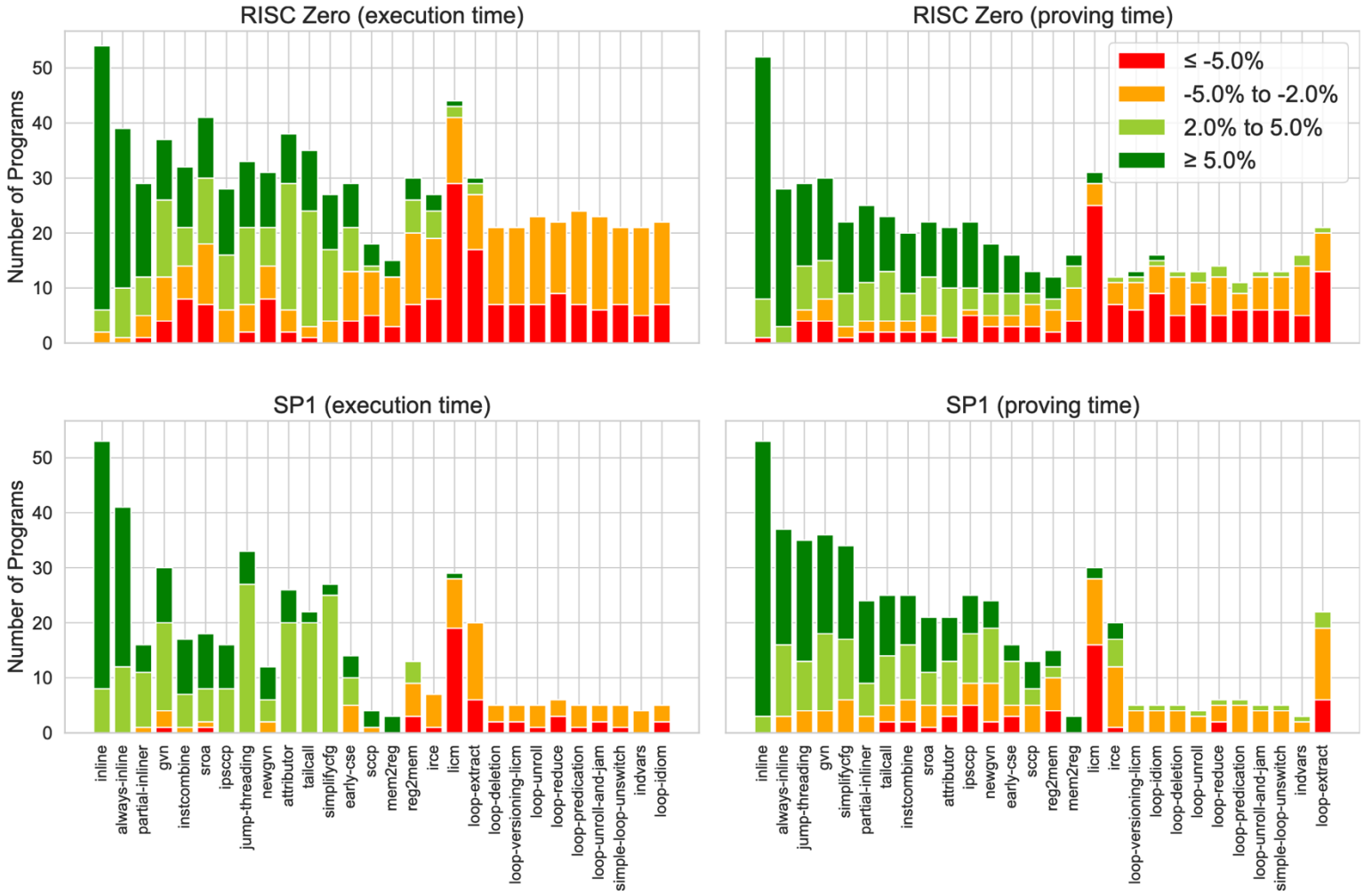
Most harmful passes

- Loop passes: `licm`, `loop-extract`, `loop-reduce`, `loop-unroll`, `irce`
- `licm` alone can slow proving by up to 2.7× on RISC Zero
- Root cause: `LCSSA` form introduces redundant `getelementptr` ops → more paging

Standard optimization levels

- `-O3` improves both execution and proving time by over 40%
- `-O0` is the worst; `-O1`/`O2`/`O3` are nearly indistinguishable on zkVMs

RQ2 · Autotuning Optimization Combinations



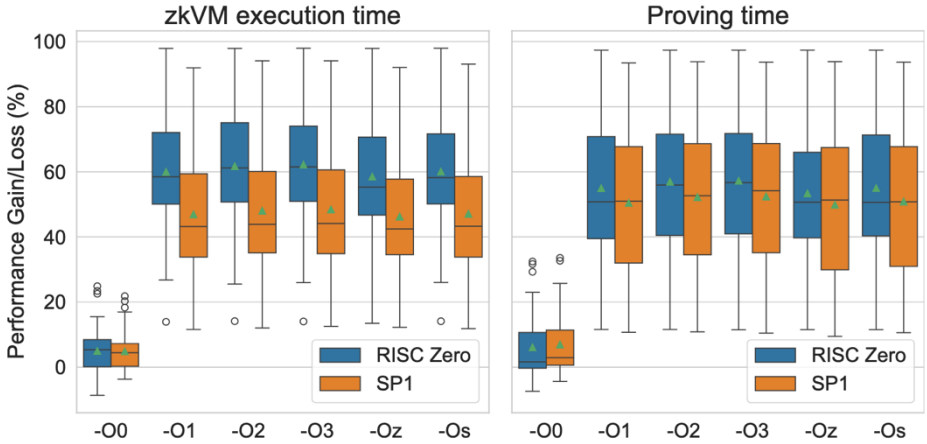
RQ2 · Autotuning Optimization Combinations

Approach

- Use `OpenTuner` (genetic search) to find optimal pass combinations
- Use cycle count as proxy (fast, noise-free) rather than proving time

Results vs. `-O3` baseline

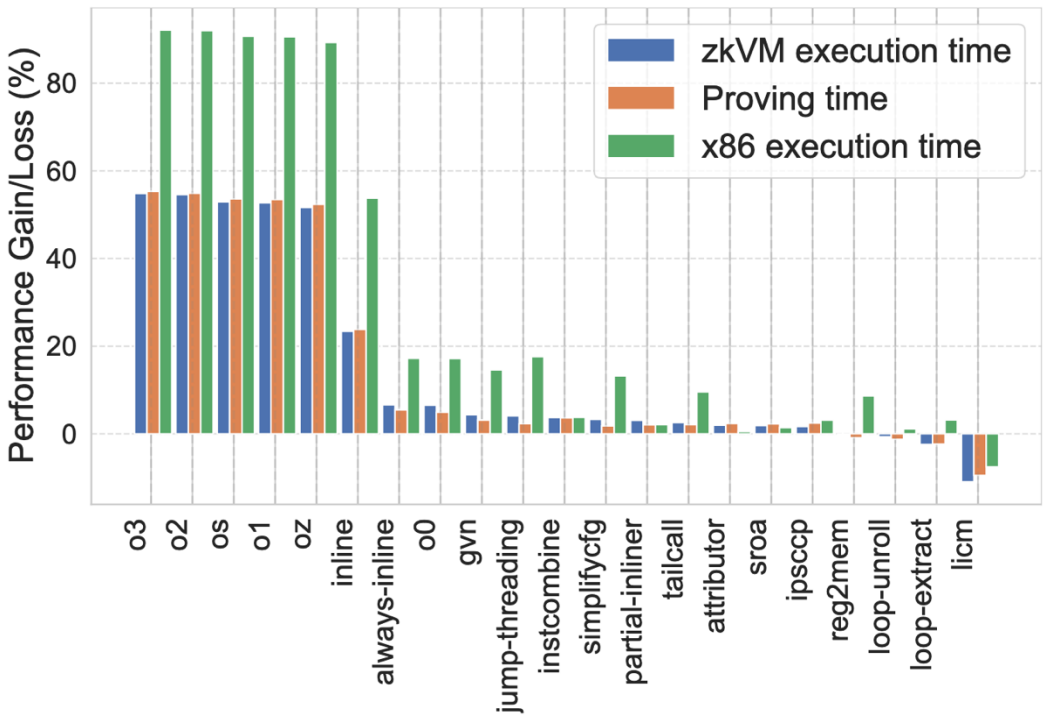
- Autotuning further improves proving time up to 2.2× beyond `-O3`
- NPB suite: up to +50% on `npb-ep`, `npb-lu`
- Crypto suite: gains smaller (already heavily precompiled)



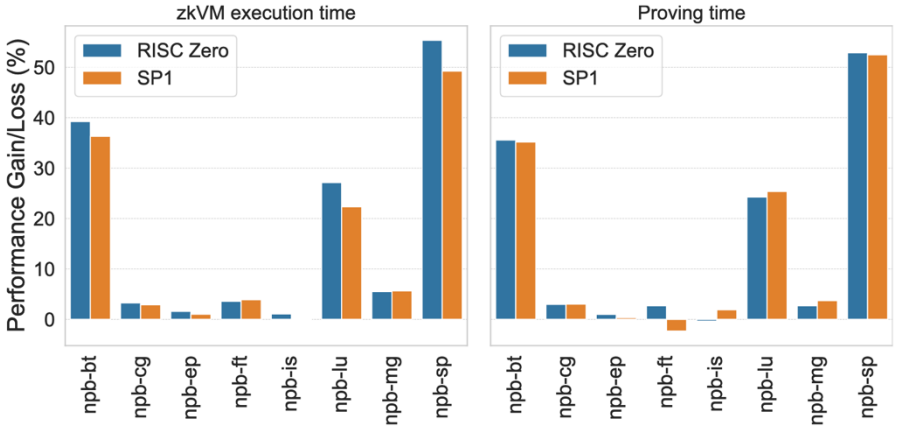
Interaction effects

- `inline` followed by `licm` → often among best sequences
- `licm` before `inline` → often among worst — ordering matters

RQ2 · Autotuning Optimization Combinations



Average impact of each optimization on zkVM and x86 performance



(a) NPB benchmark suite

(b) Crypto benchmark suite

Speedup over -O3 (with LTO)

RQ3 · Comparison with Traditional CPUs (x86)

Direction of effect is similar on x86 and zkVMs

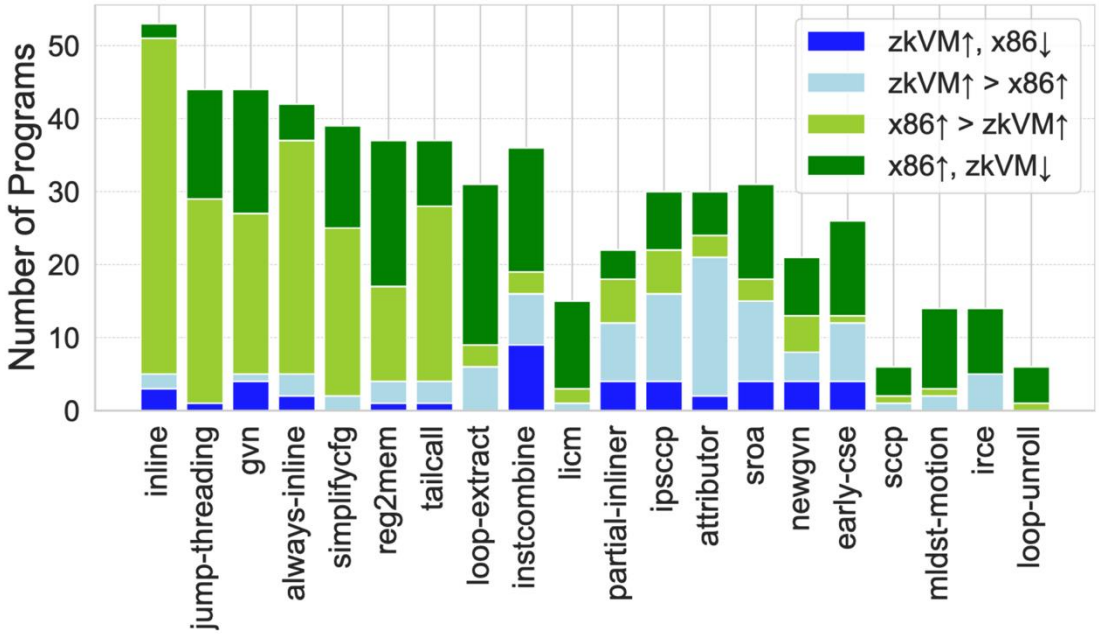
- `inline` is the top gainer on both platforms
- `licm` (in isolation) is the most detrimental on both

Magnitude is much smaller on zkVMs

- LLVM heuristics assume cache, ILP, branch prediction — absent in zkVMs
- Many passes reach only a fraction of their x86 potential

Divergent passes (help x86, hurt zkVMs)

- `reg2mem`, `loop-extract`, `mldst-motion`: hurt zkVMs in many benchmarks
- Confirms: a zkVM-specific cost model is needed



Root Causes

Two dominant zkVM cost components

- Dynamic instruction count: Kendall τ ≈ 0.87 with execution time (SP1)
- Paging cycles: specific to RISC Zero, strongly correlates with performance
- Cache hits/misses and branch mispredictions — irrelevant for zkVMs

		SP1		RISC Zero	
Performance metric	**Cost Metric**	**Kendall's τ**	**Pearson**	**Kendall's τ**	**Pearson**
zkVM exec time	Executed instr.	0.87	0.99	0.58	0.97
	Paging cycles	N/A	N/A	0.38	0.88
	Total cycles	0.87	0.99	0.59	0.97
Proving time	Executed instr.	0.49	0.9	0.5	0.95
	Paging cycles	N/A	N/A	0.33	0.89
	Total cycles	0.49	0.89	0.48	0.95
Paging cycles	Executed instr.	N/A	N/A	0.43	0.9

Case Studies

Case: `licm` on `npb-lu` benchmark

- 2.7× slower proving on RISC Zero, 1.7× on SP1
- Paging cycles: +444% on RISC Zero, +69% on SP1
- Cause: extra `getelementptr` ops from loop-invariant hoisting thrash the pager

```
const N: usize = ...;
let mut v = [0; N];
for k in 0..N {
  v[k] = 42;
}
```

(a) A simple loop of depth one.

```
const N: usize = ...;
let mut v = ...;
for k in 0..N {
 for j in 0..N {
  for i in 0..N {
   for l in 0..N {
    v[k][j][i][l] = 42;
... }
```

(b) Nested loops of depth four.

Figure 10. Simplified examples that cause significant performance regressions when using `licm`.

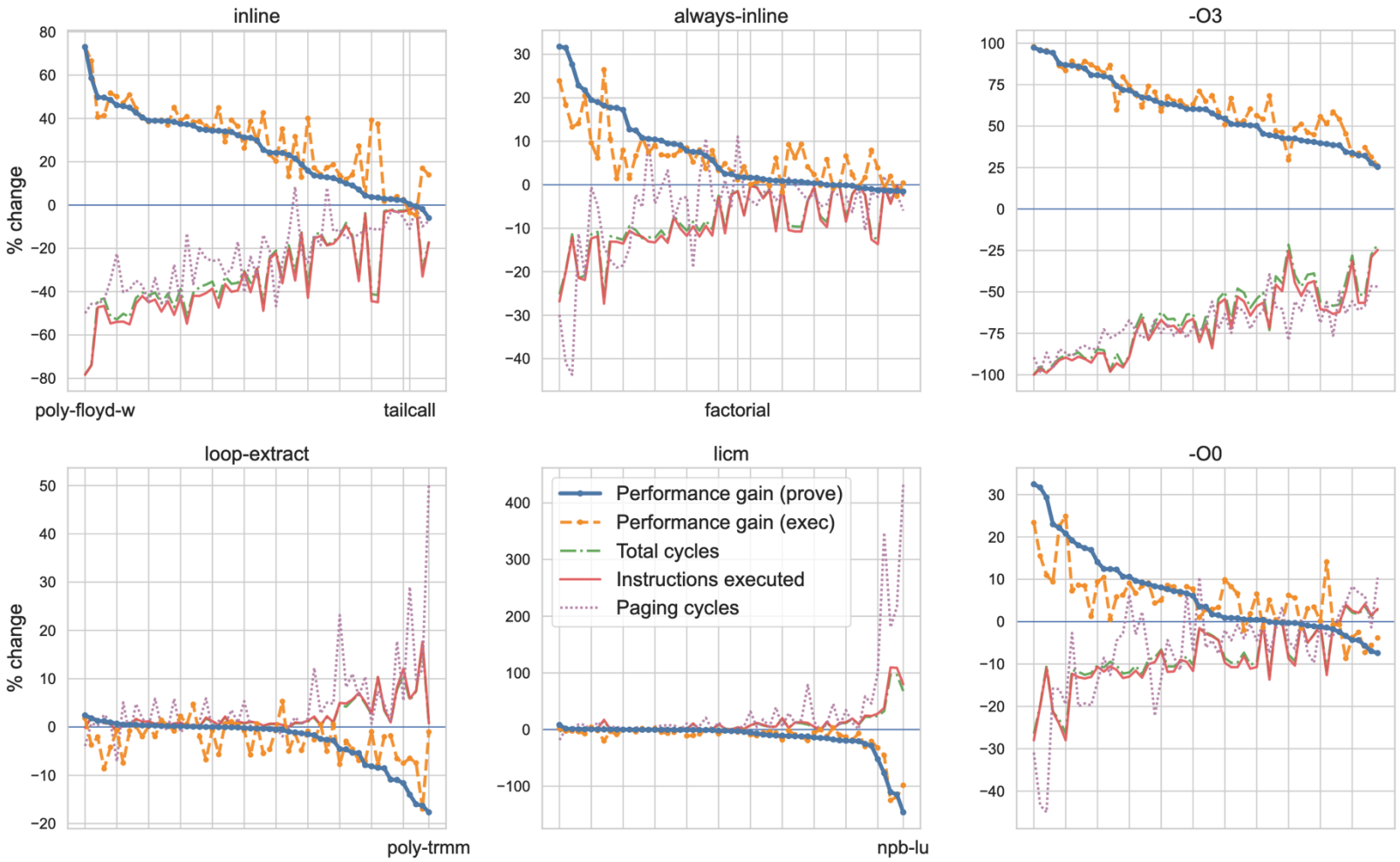
Case: aggressive `inline` on `tailcall` benchmark

- Excessive inlining → stack spills →more loads/stores → more paging
- Selective `#[inline(always)]` yields 2% faster execution without regression

```
pub fn work(x: u64) -> u64 {
  let mut sum = x;
  for j in 0..100 {
    sum = sum.wrapping_mul(31).wrapping_add(j);
  }
  sum
}
fn main() {
  let n = 1000;
  for i in 0..n { let res = work(i as u64); }
}
```

Figure 11. Example Rust function that `inline` causes a performance regression for zkVMs.

Impact of representative passes on RISC Zero performance and cost metrics



zkVM-Aware Optimizations

Three Change Sets for LLVM

Change set 1 · Adopt a zkVM-specific cost model

- Update `RISCVTTIImpl` and `RISCVTargetLowering` classes
- Mul ≈ add, `div` is cheap, memory access may trigger expensive paging

Change set 2 · Update default heuristics

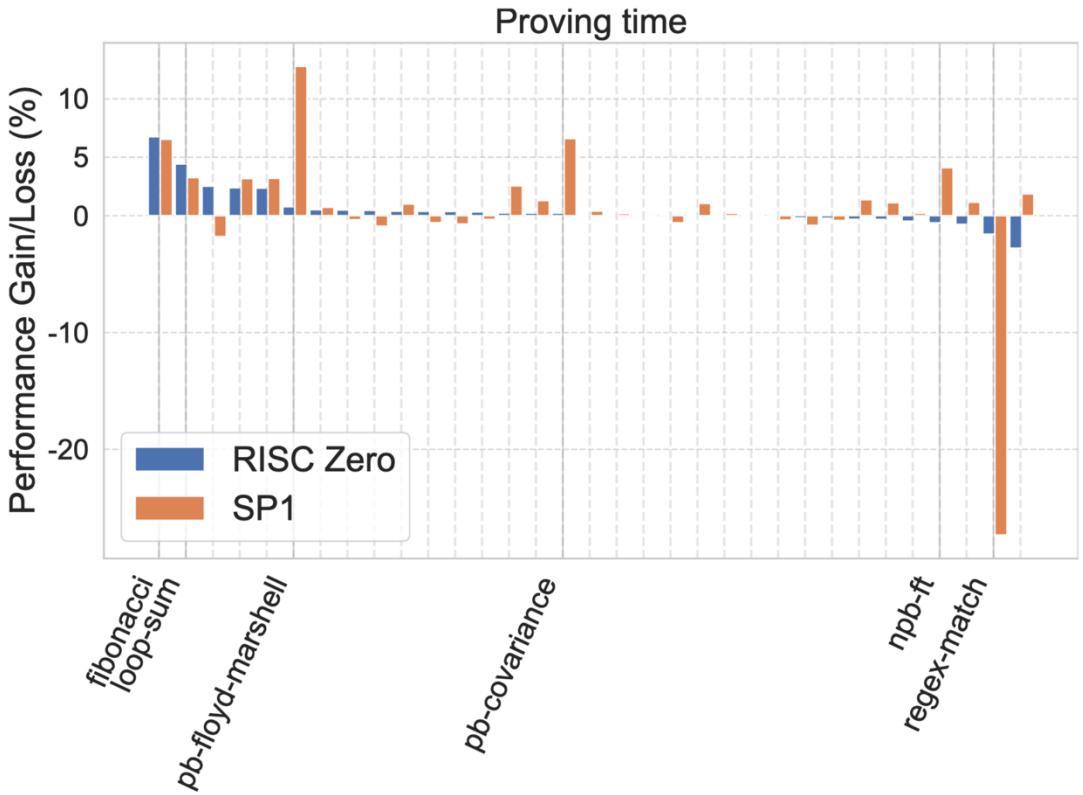
- LLVM passes employ heuristics tuned for traditional architectures, not suitable for zkVMs.
- Raise inline threshold, adjust `inline-call-penalty` (Principle 2)
- Loop-unroll only when it reduces dynamic instructions (Principle 3)
- Make `simplifycfg` branch elimination more conservative (Principle 4)

Change set 3 · Disable passes that rely on absent hardware

- Some passes offer no benefit on zkVMs, because they rely on hardware features unsupported on zkVMs.
- zkVMs execute RISC-V in order inside an arithmetic circuit — no OoO, no superscalar, no prefetcher

Total effort: < 100 LoC, a few hours of work

Results · Refinements Beat Stock -O3



Results · Refinements Beat Stock -O3

Execution time improvements

- Up to +45% on individual programs
- Average: +4.6% on RISC Zero, +1% on SP1
- Consistent, smaller gains on proving time

What drives the improvements

- Dynamic instruction count drops 2.6% on average across zkVMs
- Only 3 benchmarks show a slight regression (max +0.5%)

Caveats

- One outlier: `regex-match` on SP1 regresses 27.3% (proof-shard count change)
- Proof-of-concept — motivates a dedicated zkVM-aware LLVM backend

Conclusion & Discussion

Summary

What the paper shows

- First systematic study of LLVM passes on production zkVMs (RISC Zero, SP1)
- Default `-O3` helps (>40%), but gains are much smaller than on x86
- A few passes (`licm`, `loop-extract`, over-inlining) actively hurt zkVMs

Key insight

- zkVM performance is governed by dynamic instruction count and paging cycles
- Not cache, not ILP, not branch prediction

Takeaway for users

- Rely on LLVM `-O3` today; autotuning and zkVM-aware passes tomorrow

Limitations & Open Questions

Limitations of this study

- Only two RISC-V-based zkVMs (RISC Zero, SP1); other proof systems unexplored
- Proving-time variance on SP1 is high (closed-source, RPC-based prover)
- Refinements are proof-of-concept, not yet a production backend

Open questions left by the authors

- `instcombine` helps half the benchmarks, hurts the other half — which rewrites to keep?
- SP1's proof sharding overhead is not captured by the current cost model
- Should cost weights vary by workload (memory-heavy vs. compute-heavy)?
- Can autotuning be productized for real-time proving (e.g., Ethproofs, 12-second Ethereum blocks)?

Future Directions

A dedicated zkVM-aware LLVM backend

- Extend change sets from §5 into a full backend, not just patches
- Root-cause analysis for more passes (e.g., `instcombine`)

Profile-guided optimization & new zkVM-specific passes

- Static PGO to guide inlining and reduce instruction count
- Page-coalescing pass: pack hot variables into fewer 1 KB pages

Superoptimization

- Adapt LLVM-IR superoptimizers (e.g., `Souper`) to zkVM cost model
- Trade longer compile time for large proving-time savings

Broader scope

- Extend to other zkVMs and proof systems; unify cost metrics across the ecosystem

🙏 SC

Stefanos Chaliasos

not really it's in a stagnant stage as Thomas is busy with other stuff, and Thodoris and I, don't have time to lead this project.

would be really interested into listening in any ideas you might have

Wonjae

> **Stefanos Chaliasos**
> would be really interested into listening in any ideas you might have

Thank you! I thought about conducting a research on the making the cost model for the zkVM and also LLVM passes upon that cost model.

Stefanos Chaliasos

imo, there is untapped potential either through:

* novel, targeted, LLVM optimization passes targeting RISC-V for zkVM proving
* superoptimizations for specific programs (e.g., revm) on various inputs

heh, two different problems, I think the cost model is more difficult, especially if you want to capture many zkvms as they are quite different

if you do that, you solve many problems that even EF folks might have, e.g., metering for zkvms proving

then, I guess LLVM optimizations would be the easy part

but also solving it for a single zkvm and then design optimizations sounds very interesting

Thank You